Supplementary information for Repeated full-context calls lowered executable success in a controlled study of language model code generation

Daniil Privezentsev[1*]

[1*]National Research University Higher School of Economics, 20 Myasnitskaya Ulitsa, Moscow, 101000, Russia.

Corresponding author(s). E-mail(s): daprivezentsev@edu.hse.ru;

Supplementary comparisons and sensitivity analyses

This file preserves the source manuscript's closest-workflow comparison, execution-phase diagnostics, exploratory direct-solver contrasts, old/new task sensitivity and earlier-model corroboration. These tables do not expand the prespecified testing family. Their estimands and uncertainty are described in the main article.

Table S1 Closest code-generation workflow studies. Execution feedback means runtime test or trace outcomes returned during generation. Resource reporting includes token or call accounting. SCC feedback varies by configuration: its published compiler variant and our adapted author controller execute checks. SCC also studies role removal; our factorial intervention keeps operation sentences unchanged.

Work	Primary variation	Execution feedback	Resources	Direct baseline
SCC [1]	Role configurations and turns	Varies	Yes	Yes
MapCoder [2]	Four-role repair pipeline	Yes	Yes	Yes
AdaCoder [3]	Adaptive planning across models	Yes	Yes	Yes
PairCoder [4]	Two-agent switching across models	No	Yes	Yes
OneFlow [5]	Multi- versus single-model execution	Varies	Yes	Yes
DATS [6]	Five task-adaptive topologies	Varies	Yes	Yes
This work	Labels × one/three calls	No	Yes	Yes

Table S2 Retrospective execution-phase quality differences (percentage points). Intervals are descriptive task-bootstrap 95% intervals.

Contrast	Phase	Tasks	Repeat pairs	Difference [95% interval]
SR–SN	first	740	1096	-1.42 $[-3.54, +0.74]$
SR–SN	later	936	1855	-0.66 $[-2.23, +0.91]$
MR–MN	first	736	1090	$+0.79$ $[-1.40, +2.99]$
MR–MN	later	934	1844	$+1.41$ $[-0.11, +2.93]$
MN–SN	first	737	1092	-4.27 $[-6.56, -1.99]$
MN–SN	later	934	1846	-4.46 $[-6.37, -2.61]$
MR–SR	first	738	1092	-2.06 $[-4.27, +0.14]$
MR–SR	later	934	1849	-2.37 $[-4.00, -0.77]$

A task contributes the mean of its observed matching repeats in that phase. Task sets overlap; these estimates are not independent replications and do not change the primary Holm family.

Table S3 Exploratory direct-solver quality contrasts. Differences and intervals are percentage points; positive values favor D.

Contrast	Tasks	Difference	95% interval
D – SN	980	$+0.27$	$[-0.88, 1.43]$
D – SR	979	$+0.95$	$[-0.17, 2.11]$
D – MN	966	$+4.59$	$[2.97, 6.18]$
D – MR	970	$+3.20$	$[1.82, 4.64]$

Intervals resample tasks. These post hoc contrasts are outside the four-test Holm family and support estimation, not a new confirmatory family.

Table S4 Old/new task sensitivity for the four primary contrasts. Values are percentage points with task-bootstrap 95% intervals.

Contrast	Subset	Tasks	Difference [95% interval]
SR–SN	reused 200	194	−1.03 [−3.78, 1.72]
	new 800	788	−0.68 [−2.07, 0.72]
MR–MN	reused 200	190	+1.93 [−0.70, 4.56]
	new 800	774	+1.21 [−0.13, 2.58]
MN–SN	reused 200	192	−4.17 [−7.64, −0.87]
	new 800	776	−4.17 [−5.84, −2.58]
MR–SR	reused 200	190	−1.23 [−3.68, 1.23]
	new 800	781	−2.35 [−3.88, −0.81]

Every response was generated afresh. "Reused" means that the task ID had appeared in the earlier 200-task study; it is not reused output.

Table S5 Earlier GPT-5.4-mini corroboration on the 200 reused task IDs. Differences and descriptive intervals are percentage points; n counts complete task pairs. The four exact McNemar tests form a Holm family.

Contrast	n	Δ	95% interval	Holm p
SR–SN	191	−1.05	[−4.71, +2.62]	1.000
MR–MN	193	+1.04	[−3.11, +5.70]	1.000
MN–SN	191	−4.71	[−9.42, 0.00]	.313
MR–SR	193	−2.07	[−6.22, +2.07]	1.000

References

[1] Dong, Y., Jiang, X., Jin, Z. & Li, G. Self-collaboration code generation via ChatGPT. *ACM Transactions on Software Engineering and Methodology* **33**, 1–38 (2024). URL https://doi.org/10.1145/3672459. Preprint: arXiv:2304.07590.

[2] Islam, M. A., Ali, M. E. & Parvez, M. R. *MapCoder: Multi-agent code generation for competitive problem solving.* In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 4912–4944 (Association for Computational Linguistics, 2024). URL https://aclanthology.org/2024.acl-long.269/.

[3] Zhu, Y. *et al.* AdaCoder: An adaptive planning and multi-agent framework for function-level code generation. *IEEE Transactions on Software Engineering* **52**, 631–650 (2026). URL https://doi.org/10.1109/TSE.2025.3642621.

[4] Chen, J. *et al. PairCoder: Pair programming-inspired two-agent collaboration for code generation.* In *Findings of the Association for Computational Linguistics: ACL 2026*, 3043–3058 (Association for Computational Linguistics, 2026). URL https://aclanthology.org/2026.findings-acl.149/.

[5] Xu, J. *et al.* Rethinking the value of multi-agent workflow: A strong single agent baseline (2026). URL https://arxiv.org/abs/2601.12307. arXiv:2601.12307.

[6] Hong, Y. Learning how much to collaborate: Difficulty-aware topology selection for multi-agent code generation (2026). URL https://arxiv.org/abs/2609.13890. arXiv:2609.13890.